Lista 10

Uwaga. W Zadaniach 1 i 2 należy napisać testy jednostkowe z użyciem modułu `unittest` (w materiałach do wykładu znajduje się m.in. przykładowy zestaw testów dla klasy `TTTBoard`). Nie trzeba pisać testów do Zadania 3.

Zadanie 1 (0,75 punktu). Napisz testy jednostkowe dla metod `__init__`, `__add__`, `__mul__` i `__eq__` klasy `Complex` z Zadania 2 Listy 9.

Zadanie 2 (1 punkt + 1 punkt za testy jednostkowe). Napisz **własną**[1] klasę `Matrix` służącą do reprezentowania macierzy prostokątnych. Zaimplementuj metody:

- `__init__(self, values)` – parametr `values` to lista wierszy macierzy (każdy wiersz to lista liczb). Jeśli wiersze są różnej długości, inicjalizator ma rzucić stosowny wyjątek.
- `dim(self)` – zwraca parę liczb reprezentującą wymiary macierzy `self`.
- `transpose(self)` – zwraca macierz będącą transpozycją `self`.
- `__str__(self)` – zwraca napis reprezentujący macierz `self`.
- `__neg__(self)` – zwraca macierz odpowiadającą $-$`self`.
- `__add__(self, other_matrix)` – zwraca macierz odpowiadającą `self` + `other_matrix`. Jeśli macierze są różnych rozmiarów, metoda ma rzucić wyjątek.
- `__sub__(self, other_matrix)` – zwraca macierz odpowiadającą `self` $-$ `other_matrix`. Jeśli macierze są różnych rozmiarów, metoda ma rzucić wyjątek.
- `__mul__(self, other_matrix)` – zwraca macierz odpowiadającą `self` $\cdot$ `other_matrix`. Jeśli macierze nie są kompatybilnych wymiarów, metoda ma rzucić wyjątek.
- `__eq__(self, other_matrix)` – zwraca `True`, gdy `self` i `other_matrix` są równe, `False` w przeciwnym wypadku.

Następnie napisz testy jednostkowe **każdej** z tych metod (w tym `__init__`).

Zadanie 3 (1,25 punktu). Napisz klasę `Canvas`, której instancje reprezentują informację o kolekcji figur geometrycznych na płaszczyźnie. Klasa powinna implementować metody:

- `__init__(self)` – inicjalizator tworzący początkowo pustą kolekcję.
- `add_triangle(self, x1, y1, x2, y2, x3, y3)` – dodaje informację o trójkącie, którego wierzchołkami są punkty (`x1`, `y1`), (`x2`, `y2`), (`x3`, `y3`).
- `add_rectangle(self, x, y, height=1, width=1)` – dodaje informację o prostokącie (którego boki są równoległe do osi układu współrzędnych), którego lewym dolnym wierzchołkiem jest punkt (`x`, `y`), a którego wysokość i szerokość to `height` i `width` odpowiednio. Metoda ma rzucać wyjątek, gdy `height` lub `width` jest niedodatnie.
- `add_circle(self, x, y, r=1)` – dodaje informację o kole o promieniu `r`, którego środkiem jest punkt (`x`, `y`). Metoda ma rzucać wyjątek, jeśli `r` $\leqslant 0$.
- `draw(self, color="black")` – rysuje (z użyciem `matplotlib`) w kolorze `color` obramowania wszystkich zapamiętanych w kolekcji figur.

[1] W implementacji tej klasy nie możesz używać analogicznej klasy z bibliotek Pythona.